

Day 6 Learning

#Memory References in Java:

1. Strong Reference:

It is the default reference type in Java. Any object that has a strong reference pointing to it is not eligible for garbage collection. The JVM will rather throw an `OutOfMemoryError` than collect these objects.

Example: `StringBuilder sb = new StringBuilder();`

2. Soft Reference:

Soft references are garbage collected only when the JVM absolutely needs memory. If the heap is almost full and an `OutOfMemoryError` is about to occur, the Garbage Collector will clear soft references. Usage: Primarily used for implementing memory-sensitive caches.

Example: `SoftReference softRef = new SoftReference(new StringBuilder());`

3. Weak Reference:

Weak references do not prevent an object from being garbage collected. As soon as the Garbage Collector determines that an object is only weakly reachable, it will be cleared in the next GC cycle, regardless of the memory status. Usage: Commonly used in `WeakHashMap` to prevent memory leaks by allowing keys to be collected when they are no longer in use elsewhere.

Example: `WeakReference weakRef = new WeakReference(new StringBuilder());`

4. Phantom Reference:

Phantom references are the weakest type of reference. The `get` method of a `PhantomReference` always returns null. They are used to track when an object has been finalized and its memory is about to be reclaimed. Usage: Used for scheduling post-mortem cleanup actions or managing direct memory buffers.

Example: `PhantomReference phantom = new PhantomReference(obj, queue);`

#Enums in Java:

Key Concepts and Definitions

Enum (Enumeration): A special data type in Java used to define a fixed set of constants (like days of the week, months, departments in college, etc.). **Purpose of Enum:** Used to list fixed values, avoiding repetitive string literals or constants, which can cause errors and complicate maintenance. **Comparison with Constants:** Traditional constant definition via public static final fields is error-prone and verbose compared to enum usage.

Why Use Enums?

- Avoids duplication and spelling mistakes associated with repeated string constants.
- Enums provide type safety and centralized management of constants.
- Enums are more readable and maintainable than using multiple static final fields.
- Enums internally behave like final classes with fixed instances.

Basic Enum Syntax and Example

Enum declaration example for days of the week: `public enum Day { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY }`

- Each enum constant (e.g., SUNDAY) is an instance of the enum type (Day).
- Enum constants are comma-separated, not an array or list.

Internal Working of Enums

- Enums are final classes generated at compile time.
- Each constant is a public static final instance of the enum type.
- Each enum constant has: A name (string representation) and an ordinal (index starting from 0).

The compiler automatically generates:

- `name()` method to get the constant's name.
- `ordinal()` method to get the constant's position.

- `values()` method returning an array of all enum constants.
- `valueOf(String)` method to get enum constant from a string (throws exception if unmatched).

Using Enums in Code

- Access constants via `Day.MONDAY`, `Day.SUNDAY`, etc.
- Enums can be used in switch-case statements, with support for both old and new switch syntax (Java 12+). New switch syntax example: `switch(day) { case MONDAY -> System.out.println("Monday"); case TUESDAY -> System.out.println("Tuesday"); default -> System.out.println("Weekend"); }`
- No need for explicit break in new switch syntax; each case is an expression.

Adding Fields, Methods, and Constructors in Enums

- Enums can have fields, methods, and constructors.
- Constructors are always private or default; cannot be public.
- Example: Adding a lowercase string or a Hindi name to each day constant.
- Requires a custom constructor with parameters.
- Fields are initialized when enum constants are instantiated.
- Getters can be added to access these fields.
- Enum methods can be called on enum constants, e.g., `Day.MONDAY.getLowerCase()`.

Practical Tips

- Use enums whenever you have a fixed set of related constants.
- Avoid defining constants as strings repeatedly; centralize them in enums.
- Use `values()` method for iterating over enum constants.
- Use `valueOf()` for converting strings to enums, handling exceptions appropriately.
- Take advantage of enums' ability to have methods and fields to enrich the model.
- For nested enums, you can define enums inside classes; enums are implicitly static.

Core Insights

- Enums improve code quality by replacing error-prone constant strings or integers.
- They encapsulate related constants as a type-safe collection.

- Enums are implemented as final classes with fixed instances, ensuring immutability.
- Custom methods and fields make enums flexible and powerful beyond simple constants.
- New switch expressions simplify control flow using enums.
- Compiler-generated methods (name(), ordinal(), values()) provide utility out-of-the-box.

#Java Records:

Java Verbosity and Data Classes: Java is verbose, especially for simple data-holder classes. Data carrier classes typically store state variables (fields) and provide methods like constructors, getters, toString(), equals(), and hashCode().

Example: Traditional Java Class for Data Storage: A class Alien with two private, final fields: int id and String name.

- Requires: Constructor to initialize fields.
- Getter methods for each field.
- Overridden toString() method to print object state instead of hashcode.
- Overridden equals() and hashCode() to compare objects by value rather than memory address.
- This can lead to approximately 40 lines of code just to create a simple immutable data class.

Challenges with Traditional Approach:

- Comparing object equality by memory address returns false for different objects with the same data.
- Need manual overrides of equals() and hashCode() to compare by value.
- Verbose and repetitive boilerplate code for simple data containers.

Introduction to Java Records

What is a Record? A concise syntax to declare a class purely for immutable data storage.

Syntax example: `public record Alien(int id, String name) {}`

This single line replaces the entire traditional class with constructor, getters, toString(), equals(), and hashCode() automatically generated.

Features of Records

- **Immutable fields:** All record components (fields) are implicitly private and final.
- **Canonical constructor:** Auto-generated constructor with parameters matching components.
- **Auto-generated methods:** toString(), equals(), and hashCode() are implemented by default.
- **Accessors:** Instead of getters (getId()), record components have methods named after the components (id(), name()).
- **Inheritance:** Records cannot extend other classes (they implicitly extend java.lang.Record) but can implement interfaces.
- **Flexibility:** You can add methods (instance, static) and static variables but cannot add additional instance variables outside the record components.

Example Usage

- **Creating objects:** Alien a1 = new Alien(1, "Naveen");
- **Equality:** Equality by value works as expected: a1.equals(a2) compares values, not memory.
- **Printing:** Printing object prints field values, not hashCode.

Customizing Records

Custom Constructor: Records allow defining a canonical constructor to add logic such as input validation.

Example:

```
public Alien {  
  
    if (id == 0) throw new IllegalArgumentException("id cannot be zero");  
  
}
```

This is called a **compact canonical constructor**, where fields are not re-assigned manually — Java handles that automatically.

Default Constructor: Records do not have a default no-argument constructor by default. You can create one but must delegate to the canonical constructor with default values.

Important Notes on Records

Feature	Description
Fields	Implicitly private final and immutable
Extends	Cannot extend other classes (inherits from <code>java.lang.Record</code>)
Implements	Can implement interfaces
Methods	Can add instance, static methods and static variables
Additional Instance Variables	Not allowed outside record components
Accessor Naming	Same as component names (no get prefix)
Immutability	No setters, fields cannot be changed after construction

Benefits and Use Cases

- **Reduced boilerplate:** From ~40 lines to 1–3 lines for simple immutable data classes.
- **Readability:** Clear and concise representation of data structures.
- **Immutability:** Ensures data integrity by design, suitable for data transfer objects (DTOs).
- **Automatic method implementations:** Less error-prone than manually overriding `equals`, `hashCode`, and `toString`.

Conclusion

Java record classes provide a compact, immutable, and boilerplate-free way to define data carrier classes. They significantly reduce code verbosity while preserving the essential features of data encapsulation, equality, and string representation. Records are ideal for scenarios where objects are used solely to store data without behavior changes.

→ Interview Questions & Answers:

Q1. Kya hum ek Record ke andar instance variables declare kar sakte hain?

Answer: Nahi. Record ke andar aap extra instance fields (jaise `private int age;`) manually add nahi kar sakte. Record ka poora structure uske "header" (declaration line) par depend karta hai. Haan, aap **static variables** zaroor add kar sakte hain.

Q2. Kya ek Record kisi doosri class ko extend kar sakta hai?

Answer: Nahi. Saare Records implicitly `java.lang.Record` class ko extend karte hain. Java mein multiple inheritance allowed nahi hai, isliye Record kisi aur class ko extend nahi kar sakta. Lekin, Records **Interfaces implement kar sakte hain**.

Q3. Compact Constructor aur Canonical Constructor mein kya farq hai?

Answer:

- **Canonical Constructor:** Isme parameters wahi hote hain jo record components hain. Humein `this.id = id;` manually likhna padta hai.
- **Compact Constructor:** Isme parameters (parentheses `()`) nahi hote. Ye validation ke liye use hota hai. Iska sabse bada fayda ye hai ki assignments (`this.id = id`) Java automatically handle kar leta hai, humein likhne ki zaroorat nahi padti.

Q4. Kya Records "Mutable" ho sakte hain?

Answer: Technically, Record ke fields `final` hote hain, toh unhe re-assign nahi kiya ja sakta. Lekin, agar Record ka koi field ek **Mutable Object** hai (jaise `List` ya `Date`), toh us object ke andar ka data change kiya ja sakta hai. Isliye pure immutability ke liye humein "Defensive Copying" karni chahiye.

Q5. Records mein Accessor methods ka naam `getId()` kyun nahi hota?

Answer: Java Records modern design follow karte hain. Inka maqsad bean-style (get/set) follow karna nahi, balki "Data as Data" treat karna hai. Isliye accessor methods ka naam wahi rakha gaya hai jo component ka naam hai, jaise `id()` aur `name()`.

Q6. Kya hum Record ko `abstract` declare kar sakte hain?

Answer: Nahi. Records hamesha `final` hote hain kyunki unka structure fixed hota hai. Unhe `abstract` banana unke basic nature (data carrier) ke khilaf hai.

Q7. Record aur Enums mein kya similarity hai?

Answer: Dono hi "Special types" hain Java mein. Jaise Enum implicitly `java.lang.Enum` ko extend karta hai, waise hi Record `java.lang.Record` ko extend karta hai. Dono hi inheritance restrict karte hain aur fixed data ke liye bane hain.

#Sealed classes in Java:

Core Concepts and Definitions:

Abstract Class: A class designed to be inherited with some abstract methods; cannot be directly instantiated.

Final Class: A class that cannot be inherited by any other class; represents a final implementation.

Sealed Class: A new class type that restricts which specific classes or interfaces can extend or implement it, using the sealed keyword and a permits clause.

Sealed classes enable controlled inheritance by explicitly listing permitted subclasses or subinterfaces. Subclasses of a sealed class must be declared with one of these modifiers:

- `final` — no further inheritance allowed.
- `sealed` — further restrict inheritance and permit specific subclasses.
- `non-sealed` — open for unrestricted inheritance.

For interfaces, subclasses can only be sealed or non-sealed (interfaces cannot be final).

How Sealed Classes Work

When declaring a sealed class, use the syntax: `public sealed class A permits B, C { // class body }`

This restricts inheritance of class A exclusively to classes B and C. Attempting to extend a sealed class by a class not listed in the permits clause results in a compile-time error: error: class D is not allowed to extend sealed class A. The permitted subclasses (B, C) must themselves be declared as final, sealed, or non-sealed.

Example Hierarchy and Modifiers

Class A (sealed): Base class permitting only B and C to extend it. **Class B (non-sealed):** Can be freely extended by other classes (like D). **Class C (final):** No further inheritance allowed beyond C. **Class D (Normal):** Extends B successfully; cannot extend A directly.

If B is declared final, no other class can inherit from B. If B is declared sealed, it must specify which subclasses can inherit it. If B is declared non-sealed, it is open to unrestricted subclassing.

Sealed Classes and Interfaces

Sealed interfaces also follow similar principles:

- Use sealed interface X permits Y to restrict which interfaces can extend X.
- Subinterfaces must be declared as sealed or non-sealed.
- Interfaces cannot be declared final.

Practical Use Cases

Sealed classes are useful in application design where restricted inheritance models are desired. Examples include:

- Defining a payment gateway class hierarchy where only specific gateway types are allowed.
- Defining a Computer class where only subclasses like Laptop, Desktop, and Mobile are permitted.
- This feature improves code maintainability and security by explicitly controlling subclassing.

Key Insights

- Sealed classes provide finer control over class inheritance than final or abstract classes alone.

- The permits clause is mandatory to list allowed subclasses or subinterfaces.
- Subclasses of a sealed class must explicitly state their inheritance mode (final, sealed, or non-sealed).
- Sealed classes do not eliminate inheritance but restrict it to a controlled set.
- This feature is optional and should be used when such restrictions fit the project design.

Conclusion

Sealed classes in Java introduce a powerful mechanism for controlling class and interface inheritance, allowing developers to restrict subclassing to a predefined set of permitted classes or interfaces. This enhances design clarity, reduces errors, and improves maintainability for complex Java applications. While not mandatory for every project, sealed classes provide an important option for scenarios requiring strict inheritance rules.

→ Interview Questions & Answers:

Q1. Kya ek Sealed Class ke permitted subclasses ka same file mein hona zaroori hai? Answer: Nahi, ye zaroori nahi hai. Agar subclasses same file mein hain, toh aap "permits" clause ko skip kar sakte hain, Java use automatically detect kar lega. Lekin agar subclasses alag files mein hain, toh wo same package (ya same module) mein honi chahiye aur "permits" clause ka use lazmi hai.

Q2. Kya hum ek "non-sealed" class ko wapis "sealed" bana sakte hain? Answer: Haan, bilkul. Agar ek class (A) sealed hai aur uski subclass (B) non-sealed hai, toh B ko normal class ki tarah treat kiya jayega. Lekin agar aap chahein toh B ko "sealed" declare karke uske aage ki inheritance ko restrict kar sakte hain. Iska matlab hai ki inheritance chain mein control wapis laya ja sakta hai.

Q3. "permits" clause kab optional hota hai? Answer: "permits" clause tab optional hota hai jab saari permitted subclasses usi same source file (.java file) ke andar defined honi chahiye jahan sealed class defined hai. Is case mein compiler khud hi subclasses ko pehchan leta hai.

Q4. Sealed Classes aur Final Classes mein kya main difference hai? Answer: Final class inheritance ko puri tarah se band (block) kar deti hai—koi bhi use extend nahi kar sakta. Jabki Sealed class inheritance ko "restrict" karti hai—sirf wahi classes extend kar sakti hain jinhe permission di gayi hai.

Q5. Kya ek interface "final" ho sakta hai? Sealed interfaces ke context mein samjhayein. Answer: Nahi, interface kabhi final nahi ho sakta kyunki uska maqsad

implement hona hai. Lekin Java 17 mein hum interfaces ko "sealed" bana sakte hain. Ek sealed interface ye restrict karta hai ki kaunsi classes ya interfaces use implement/extend kar sakti hain. Iske subclasses ya toh "sealed" honi chahiye ya "non-sealed".

Q6. Kya Reflection API sealed classes ke restrictions ko tod sakta hai? Answer:

Nahi, Java ka Reflection API sealed classes ke rules ka samman karta hai. Aap runtime par reflection use karke kisi aisi class ko subclass nahi bana sakte jo permits list mein nahi hai. `Class.isSealed()` aur `Class.getPermittedSubclasses()` jaise naye methods bhi add kiye gaye hain reflection support ke liye.

Q7. Sealed classes "Pattern Matching" (switch expressions) mein kaise help karti hain? Answer:

Ye sabse bada fayda hai. Jab aap `switch` mein sealed class ka use karte hain, toh compiler ko pata hota hai ki total kitni subclasses hain. Isliye aapko "default" case likhne ki zaroorat nahi padti (Exhaustiveness check), kyunki compiler verify kar sakta hai ki saare possible cases cover ho gaye hain.

#Optional in Java:

- **Optional Class:** A container object introduced in Java 8 (`java.util` package) used to contain not-null objects. It represents a value that may or may not be present.
- **Purpose of Optional:** The primary design goal is to provide a type-level solution for representing optional values instead of using null references. This helps in avoiding `NullPointerException` (NPE) and makes the code more readable and expressive.

Why Use Optional?

- **Explicit API Design:** When a method returns `Optional`, it explicitly tells the caller that the value might be missing, forcing them to handle the empty case.
- **Avoids Null Checks:** It reduces the need for repetitive "if (obj != null)" blocks throughout the code.
- **Functional Style:** It integrates perfectly with Java Streams and Lambda expressions for cleaner data processing.

Commonly Used Methods:

Creating Optional Objects:

- **Optional.of(value):** Creates an Optional with a non-null value. Throws NPE if the value is null.
- **Optional.ofNullable(value):** Creates an Optional that can hold a value or be empty if the input is null.
- **Optional.empty():** Returns an empty Optional instance.

Accessing Values Safely:

- **isPresent() / isEmpty():** Returns true/false based on whether a value is inside the container.
- **ifPresent(Consumer):** Executes a block of code only if the value is present.
- **orElse(defaultValue):** Returns the value if present, otherwise returns the specified default value.
- **orElseThrow():** Returns the value or throws an exception if the container is empty.
- **map() / flatMap():** Used to transform the value inside the Optional if it exists.

Best Practices and Restrictions:

Fields and Parameters: Optional should not be used for class fields or method parameters. It is primarily intended for use as a method return type. Using it as a field can cause issues with Serialization.

Collection Handling: Never return an Optional of a Collection (e.g., Optional<List>). Instead, return an empty Collection to represent the absence of data.

Performance: While Optional adds a small overhead (as it is an extra object), the benefit of code safety usually outweighs the performance cost in standard applications.

Comparison Table:

Aspect	Traditional Null	Java Optional
Safety	High risk of NullPointerException	Much safer; forces handling of empty cases

Intent	Ambiguous (Is it null due to error or data?)	Clear (Data might be intentionally absent)
Readability	Cluttered with null checks	Clean and declarative (functional style)
Usage	Anywhere	Recommended only for Return Types

Conclusion:

The **Optional class is a final Class in Java used as object containers** that shifts the burden of null handling from the runtime to the compile-time (via API design). By treating "absence of value" as a first-class concept, developers can write more robust, maintainable, and crash-proof Java applications, especially when working with modern features like Streams and Spring Boot Data JPA.